We're also doing something else new here: using a form that isn't directly associated with a model object. To see how that works, let's look at the template for the `sessions#new` action:

rails7/depot_r/app/views/sessions/new.html.erb

```
<div class="mx-auto md:w-2/3 w-full">
  <% if notice.present? %>
    <p class="py-2 px-3 bg-green-50 mb-5 text-green-500 font-medium
              rounded-lg inline-block" id="notice">
      <%= notice %>
    </p>
  <% end %>

  <%= form_tag do %>
    <h2 class="font-bold text-3xl">Please Log In</h2>

    <div class="my-5">
      <%= label_tag :name, 'Name:' %>
      <%= text_field_tag :name, params[:name], class: "payment-field" %>
    </div>

    <div class="my-5">
      <%= label_tag :password, 'Password:' %>
      <%= password_field_tag :password, params[:password],
        class: "payment-field" %>
    </div>

    <div class="actions">
      <%= submit_tag "Login",  class: "rounded-lg py-3 px-5
        bg-green-600 text-black inline-block font-medium cursor-pointer" %>
    </div>
  <% end %>
</div>
```

This form is different from ones you saw earlier. Rather than using `form_with`, it uses `form_tag`, which simply builds a regular HTML `<form>`. Inside that form, it uses `text_field_tag` and `password_field_tag`, two helpers that create HTML `<input>` tags. Each helper takes two parameters. The first is the name to give to the field, and the second is the value with which to populate the field. This style of form allows us to associate values in the `params` structure directly with form fields—no model object is required. In our case, we choose to use the `params` object directly